Failure (force the parser to actually fail) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `structured_output_practice.py` (Failure section): you need to know, before it happens in production, exactly which exception a bad reply raises — and that depends on *how* you asked for structure. The "shape during" approach (`.with_structured_output()`) constrains the model as it decodes, so a wrong shape almost never gets through and you rarely see it fail. This exercise deliberately uses the other approach, "check after" — a `PydanticOutputParser` that takes the model's free-text reply and validates it afterwards — because that's where a reply in the wrong shape *really does* raise, as an `OutputParserException`. Triggering it on purpose here means you've seen the exact error, and its message, before you ever have to catch it for real. **If not:** the Build Task's "a reply that doesn't fit the shape raises a clear, named error" test would check an error you had never seen fire.

Solution

```
# structured_output_practice.py — Failure section
from pydantic import BaseModel
from langchain_core.exceptions import OutputParserException
from langchain_core.output_parsers import PydanticOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI


class Rating(BaseModel):
    score: int


parser = PydanticOutputParser(pydantic_object=Rating)
model = ChatOpenAI(model="gpt-4o-mini", temperature=0)


# Build and run a rating chain for one template. If the template has a
# {format_instructions} slot, fill it once so the caller only passes {thing}.
# Returns a Rating, or None when the model's text won't parse.
def get_rating(template, thing):
    prompt = ChatPromptTemplate.from_template(template)
```

```
    if "{format_instructions}" in template:
        prompt = prompt.partial(
            format_instructions=parser.get_format_instructions()
        )
    try:
        return (prompt | model | parser).invoke({"thing": thing})
    except OutputParserException:
        return None          # catch the parser's own error, by name


# case 1: no instructions -> the model answers in a sentence that won't parse.
no_hint = get_rating("Rate {thing} from 1 to 10.", "pizza")
print(f"case 1 (no instructions)   -> {no_hint}")

# case 2: with instructions -> the model returns parseable JSON.
fixed = "Rate {thing} from 1 to 10.\n{format_instructions}"
with_hint = get_rating(fixed, "pizza")
print(f"case 2 (with instructions) -> {with_hint}")

# case 3: parse plain text by hand — no API call needed.
try:
    parser.parse("I'd say eight")
    print("case 3 (parse by hand)     -> accepted (wrong!)")
except OutputParserException:
    print("case 3 (parse by hand)     -> OutputParserException")
```

Expected output (the score varies):

```
case 1 (no instructions)   -> None
case 2 (with instructions) -> score=9
case 3 (parse by hand)     -> OutputParserException
```

Note: the Build Task's test file uses the same `parser.parse(...)` check for its "wrong shape" case.